

IF2224 Teori Bahasa Formal dan Otomata

Milestone 3



13524121 - Billy Ontoseno Irawan

13524122 - Nathaniel Christian

13524139 - Azri Arzaq Pohan

13524142 - Rasyad Satyatma

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026**

BAB 1

Dasar Teori

1.1 *Attributed Grammar*

Attributed grammar adalah perluasan dari *context-free grammar* (CFG) dengan penambahan atribut pada setiap simbol terminal maupun non-terminal serta aturan semantik yang menentukan bagaimana atribut tersebut digunakan. Atribut dibagi menjadi dua:

- *Synthesized attribute* yang nilainya dihitung dari atribut anak-anak sebuah node dan mengalir naik ke atas pohon
- *Inherited attribute* yang dihitung dari parent ke anak, memungkinkan evaluasi semantik selama parsing.

L-attributed grammar merupakan subset dari *attributed grammar* yang membatasi cara atribut dihitung dengan satu kali *traversal* kiri-ke-kanan. *Synthesized attribute* dihitung setelah kunjungan ke anak-anak, sedangkan *inherited attribute* dihitung saat menelusuri produksi dari kiri ke kanan

1.2 *Symbol Table*

Symbol table adalah struktur data yang menyimpan informasi tentang identifier dalam suatu program seperti nama variabel, konstanta, tipe, prosedur/fungsi, beserta metadata yang relevan seperti tipe, lokasi deklarasi (scope/level), dan informasi tambahan seperti ukuran, alamat/offset, atau pointer ke struktur tipe komposit

1.3 *Abstract Syntax Tree*

Abstract Syntax Tree (AST) adalah representasi pohon abstrak dari struktur program yang menekankan konstruksi semantik dan mengabaikan detail sintaksis yang tidak relevan seperti tanda baca atau urutan aturan produksi spesifik. AST berbeda dari *parse tree* yang cenderung memuat semua simbol non-terminal dan terminal sesuai derivasi dari grammar. AST hanya menyimpan struktur yang relevan untuk analisis dan generasi kode; operator direpresentasikan sebagai node internal dengan operand sebagai anak.

Decorated AST adalah AST yang telah diberikan anotasi semantik. Setiap node yang relevan menyimpan atribut tambahan, seperti tipe hasil ekspresi (integer, real, boolean, dan sebagainya), indeks entri di *symbol table*, informasi blok atau level leksikal, dan parameter lain yang diperlukan untuk tahap generasi kode

1.4 *Semantic Analysis*

Semantic analysis merupakan fase setelah *lexical analysis* dan *syntax analysis* yang bertugas memverifikasi makna semantik program menggunakan *attributed grammar* beserta *symbol table* dengan input *parse tree* hasil parser dan menghasilkan Decorated AST. Fase ini memastikan bahwa deklarasi dan pernyataan program benar secara semantik.

Semantic analysis berfokus pada *type checking* yang memeriksa apakah operand operasi sesuai definisi tipe, *flow control check* yang mengkonfirmasi penggunaan struktur kontrol seperti break atau continue berada di tempat yang benar, pengecekan deklarasi variabel, serta pengecekan lingkup atau scope variabel yang digunakan. *Semantic analyzer* mengumpulkan informasi tipe dan menyimpannya di *symbol table*. Informasi ini kemudian digunakan untuk *intermediate code generation*. Proses ini membedakan antara *static semantics* yang dicek saat kompilasi dan *dynamic semantics* yang dievaluasi saat runtime.

BAB 2

Perancangan dan Implementasi

Pada Milestone 3, compiler Arion ditambahkan tahap semantic analysis yang berjalan setelah lexer dan parser. Tahap ini menerima parse tree hasil parser, kemudian melakukan translasi menjadi Decorated Abstract Syntax Tree (Decorated AST) sekaligus membangun dan memvalidasi symbol table. Implementasi tahap semantic analysis memiliki tiga komponen utama, yaitu struktur AST, symbol table, dan semantic analyzer.

2.1 Struktur AST

Kelas AST dan turunannya diimplementasikan pada *AST.hpp* dan *AST.cpp*. Setiap node AST menyimpan atribut semantik berupa type, level, tabRef, dan line. Atribut ini digunakan untuk mendekorasi node AST, sehingga AST yang dihasilkan sudah dilengkapi informasi tipe data, level scope, dan referensi ke symbol table. Konversi Parse Tree menjadi AST dilakukan dengan Syntax-Directed Translation.

AST yang dibentuk merupakan versi ringkas dari parse tree. Informasi sintaksis seperti keyword begin, end, semicolon, then, dan do tidak disimpan sebagai node utama karena tidak diperlukan pada tahap semantic analysis. Node AST hanya menyimpan struktur yang bermakna secara semantik, seperti assignment, deklarasi, ekspresi, operator, variabel, pemanggilan prosedur/fungsi, dan struktur kontrol.

Node yang diimplementasikan mencakup ProgramNode, BlockNode, VarDeclNode, ConstDeclNode, TypeDeclNode, AssignNode, ProcCallNode, IfNode, WhileNode, ForNode, RepeatNode, CaseNode, BinOpNode, UnaryOpNode, VarNode, ArrayAccessNode, RecordAccessNode, FuncCallNode, ArrayTypeNode, RecordTypeNode, RangeNode, dan EnumeratedNode. Berikut pengimplementasiannya.

```
(inside) ./src/Semantic/AST.hpp
```

```
#ifndef __AST_HPP__
#define __AST_HPP__

#include <string>
#include <vector>
#include <iostream>
#include <sstream>

class ASTNode;

class ASTNode {
public:
    std::string type;
    int level;
    int tabRef;
    int line;
```

```

    std::vector<ASTNode *> children;

    ASTNode();
    virtual ~ASTNode();

    virtual std::string nodeName() const = 0;

    virtual std::string annotation() const;

    virtual std::string label() const;

    void addChild(ASTNode *child);

    void print(std::ostream &out, const std::string &prefix = "", bool isLast
= true) const;
};

class ProgramNode : public ASTNode {
public:
    std::string name;
    ASTNode *declarations;
    ASTNode *body;

    ProgramNode(const std::string &n, ASTNode *decls, ASTNode *blk);
    std::string nodeName() const override { return "ProgramNode"; }
    std::string label() const override;
    std::string annotation() const override;
};

class BlockNode : public ASTNode {
public:
    int blockIndex;

    BlockNode(ASTNode *stmts = nullptr);
    std::string nodeName() const override { return "BlockNode"; }
    std::string annotation() const override;
};

class DeclarationListNode : public ASTNode {
public:
    DeclarationListNode();
    std::string nodeName() const override { return "DeclarationListNode"; }

    void addDeclaration(ASTNode *decl);
};

class VarDeclNode : public ASTNode {
public:
    std::string varName;
    std::string typeName;
    ASTNode *typeNode;

    VarDeclNode(const std::string &name, const std::string &tname, ASTNode
*tnode = nullptr);
    std::string nodeName() const override { return "VarDecl"; }
};

```

```

        std::string label() const override;
        std::string annotation() const override;
};

class ConstDeclNode : public ASTNode {
public:
    std::string constName;
    ASTNode *value;

    ConstDeclNode(const std::string &name, ASTNode *val);
    std::string nodeName() const override { return "ConstDecl"; }
    std::string label() const override;
    std::string annotation() const override;
};

class TypeDeclNode : public ASTNode {
public:
    std::string typeName;
    ASTNode *typeDef;

    TypeDeclNode(const std::string &name, ASTNode *def);
    std::string nodeName() const override { return "TypeDecl"; }
    std::string label() const override;
    std::string annotation() const override;
};

class AssignNode : public ASTNode {
public:
    ASTNode *target;
    ASTNode *value;

    AssignNode(ASTNode *tgt, ASTNode *val);
    std::string nodeName() const override { return "Assign"; }
    std::string label() const override;
    std::string annotation() const override;
};

class ProcCallNode : public ASTNode {
public:
    std::string procName;
    std::vector<ASTNode *> args;

    ProcCallNode(const std::string &name, const std::vector<ASTNode *>
&arguments);
    std::string nodeName() const override { return "ProcCall"; }
    std::string label() const override;
    std::string annotation() const override;
};

class IfNode : public ASTNode {
public:
    ASTNode *condition;
    ASTNode *thenStmt;
    ASTNode *elseStmt;

    IfNode(ASTNode *cond, ASTNode *thenS, ASTNode *elseS = nullptr);

```

```

        std::string nodeName() const override { return "IfNode"; }
        std::string annotation() const override;
};

class WhileNode : public ASTNode {
public:
    ASTNode *condition;
    ASTNode *body;

    WhileNode(ASTNode *cond, ASTNode *bdy);
    std::string nodeName() const override { return "WhileNode"; }
    std::string annotation() const override;
};

class ForNode : public ASTNode {
public:
    std::string loopVar;
    std::string direction;
    ASTNode *start;
    ASTNode *end;
    ASTNode *body;

    ForNode(const std::string &var, const std::string &dir,
            ASTNode *st, ASTNode *en, ASTNode *bdy);
    std::string nodeName() const override { return "ForNode"; }
    std::string label() const override;
    std::string annotation() const override;
};

class RepeatNode : public ASTNode {
public:
    ASTNode *body;
    ASTNode *condition;

    RepeatNode(ASTNode *bdy, ASTNode *cond);
    std::string nodeName() const override { return "RepeatNode"; }
    std::string annotation() const override;
};

class CaseBranchNode : public ASTNode {
public:
    std::vector<ASTNode *> labels;
    ASTNode *statement;

    CaseBranchNode(const std::vector<ASTNode *> &lbls, ASTNode *stmt);
    std::string nodeName() const override { return "CaseBranch"; }
};

class CaseNode : public ASTNode {
public:
    ASTNode *selector;
    std::vector<CaseBranchNode *> branches;

    CaseNode(ASTNode *sel, const std::vector<CaseBranchNode *> &brs);
    std::string nodeName() const override { return "CaseNode"; }
    std::string annotation() const override;
};

```

```

};

class EmptyNode : public ASTNode {
public:
    EmptyNode();
    std::string nodeName() const override { return "EmptyNode"; }
};

class BinOpNode : public ASTNode {
public:
    std::string op;

    ASTNode *left;
    ASTNode *right;

    BinOpNode(const std::string &oper, ASTNode *l, ASTNode *r);
    std::string nodeName() const override { return "BinOp"; }
    std::string label() const override;
    std::string annotation() const override;
};

class UnaryOpNode : public ASTNode {
public:
    std::string op;
    ASTNode *operand;

    UnaryOpNode(const std::string &oper, ASTNode *opnd);
    std::string nodeName() const override { return "UnaryOp"; }
    std::string label() const override;
    std::string annotation() const override;
};

class VarNode : public ASTNode {
public:
    std::string name;

    VarNode(const std::string &n);
    std::string nodeName() const override { return "Var"; }
    std::string label() const override;
    std::string annotation() const override;
};

class ArrayAccessNode : public ASTNode {
public:
    ASTNode *array;
    ASTNode *index;

    ArrayAccessNode(ASTNode *arr, ASTNode *idx);
    std::string nodeName() const override { return "ArrayAccess"; }
    std::string annotation() const override;
};

class RecordAccessNode : public ASTNode {
public:
    ASTNode *record;
    std::string field;

```

```

    RecordAccessNode(ASTNode *rec, const std::string &fld);
    std::string nodeName() const override { return "RecordAccess"; }
    std::string label() const override;
    std::string annotation() const override;
};

class NumberNode : public ASTNode {
public:
    int value;

    NumberNode(int val);
    std::string nodeName() const override { return "Number"; }
    std::string label() const override;
    std::string annotation() const override;
};

class RealNode : public ASTNode {
public:
    double value;

    RealNode(double val);
    std::string nodeName() const override { return "Real"; }
    std::string label() const override;
    std::string annotation() const override;
};

class BooleanNode : public ASTNode {
public:
    bool value;

    BooleanNode(bool val);
    std::string nodeName() const override { return "Boolean"; }
    std::string label() const override;
    std::string annotation() const override;
};

class StringNode : public ASTNode {
public:
    std::string value;

    StringNode(const std::string &val);
    std::string nodeName() const override { return "String"; }
    std::string label() const override;
    std::string annotation() const override;
};

class CharNode : public ASTNode {
public:
    char value;

    CharNode(char val);
    std::string nodeName() const override { return "Char"; }
    std::string label() const override;
    std::string annotation() const override;
};

```

```

class FuncCallNode : public ASTNode {
public:
    std::string funcName;
    std::vector<ASTNode *> args;

    FuncCallNode(const std::string &name, const std::vector<ASTNode *>
&arguments);
    std::string nodeName() const override { return "FuncCall"; }
    std::string label() const override;
    std::string annotation() const override;
};

class SubprogramDeclNode : public ASTNode {
public:
    std::string kind;
    std::string subName;
    std::string returnType;
    std::vector<VarDeclNode *> params;
    ASTNode *body;

    SubprogramDeclNode(const std::string &k, const std::string &n,
                        const std::vector<VarDeclNode *> &p,
                        const std::string &retType, ASTNode *bdy);
    std::string nodeName() const override { return "SubprogramDecl"; }
    std::string label() const override;
    std::string annotation() const override;
};

class ArrayTypeNode : public ASTNode {
public:
    ASTNode *indexType;
    ASTNode *elementType;

    ArrayTypeNode(ASTNode *idxType, ASTNode *elemType);
    std::string nodeName() const override { return "ArrayType"; }
    std::string annotation() const override;
};

class RecordTypeNode : public ASTNode {
public:
    std::vector<VarDeclNode *> fields;

    RecordTypeNode(const std::vector<VarDeclNode *> &flds);
    std::string nodeName() const override { return "RecordType"; }
    std::string annotation() const override;
};

class RangeNode : public ASTNode {
public:
    ASTNode *low;
    ASTNode *high;

    RangeNode(ASTNode *l, ASTNode *h);
    std::string nodeName() const override { return "Range"; }
    std::string annotation() const override;
};

```

```

};

class EnumeratedNode : public ASTNode {
public:
    std::vector<std::string> values;

    EnumeratedNode(const std::vector<std::string> &vals);
    std::string nodeName() const override { return "Enumerated"; }
    std::string label() const override;
    std::string annotation() const override;
};

namespace ASTHelper {
    inline std::string formatAnnotation(const std::string &type, int level,
int tabRef) {
        std::ostringstream oss;
        bool hasAny = false;
        oss << " -> ";
        if (!type.empty() && type != "void") {
            oss << "type:" << type;
            hasAny = true;
        }
        if (level > 0) {
            if (hasAny)
                oss << ", ";
            oss << "lev:" << level;
            hasAny = true;
        }
        if (tabRef > 0) {
            if (hasAny)
                oss << ", ";
            oss << "tab_index:" << tabRef;
            hasAny = true;
        }
        if (!hasAny)
            return "";
        return oss.str();
    }
}

#endif

```

(inside) ./src/Semantic/AST.cpp

(source code terlalu panjang; bisa dibaca dari file)

2.2 Symbol Table

Kelas symbol table diimplementasikan pada *SymbolTable.hpp* dan *SymbolTable.cpp*. Struktur symbol table mengikuti spesifikasi dengan tiga tabel utama, yaitu tab, btab, dan atab. tab menyimpan identifier seperti reserved word, tipe, konstanta, variabel, prosedur, fungsi, dan

parameter. btab menyimpan informasi blok scope, termasuk pointer ke identifier terakhir, parameter terakhir, ukuran parameter, dan ukuran variabel lokal. atab menyimpan informasi tipe array seperti tipe indeks, tipe elemen, batas bawah, batas atas, ukuran elemen, dan ukuran total array.

Pengelolaan scope dilakukan menggunakan display, dengan operasi pushScope() saat masuk scope baru dan popScope() saat keluar scope. Definisi struktur ketiga tab ditunjukkan pada potongan kode berikut.

```
(inside) ./src/Semantic/SymbolTable.hpp
```

```
struct TabEntry {
    std::string id;
    std::string obj;

    int type;
    int ref;

    int nrm;
    int lev;
    int adr;

    int link;

    TabEntry() : type(0), ref(0), nrm(1), lev(0), adr(0), link(0) {}
    TabEntry(const std::string &i, const std::string &o, int t, int r,
              int n, int l, int a, int lk)
        : id(i), obj(o), type(t), ref(r), nrm(n), lev(l), adr(a), link(lk) {}
};

struct BtabEntry {
    int last;
    int lpar;
    int psze;
    int vsze;

    BtabEntry() : last(0), lpar(0), psze(0), vsze(0) {}
    BtabEntry(int lst, int lp, int ps, int vs)
        : last(lst), lpar(lp), psze(ps), vsze(vs) {}
};

struct AtabEntry {
    int arrays;
    int xtyp;
    int etyp;
    int eref;
    int low;
    int high;
    int elsz;
    int size;

    AtabEntry() : arrays(0), xtyp(0), etyp(0), eref(0),
                  low(0), high(0), elsz(0), size(0) {}
};
```

```

    AtabEntry(int a, int xt, int et, int er, int lo, int hi, int es, int sz)
        : arrays(a), xtyp(xt), etyp(et), eref(er),
          low(lo), high(hi), elsz(es), size(sz) {}
};

```

Sebelum semantic analysis berjalan, symbol table diinisialisasi dengan reserved word dan predefined identifier. Predefined identifier yang terdefinisi adalah sebagai berikut.

Identifier	Object dalam tab	Type	Keterangan
Integer	type	Integer	Tipe bilangan bulat.
Real	type	Real	Tipe bilangan pecahan.
Boolean	type	Boolean	Tipe logika.
Char	type	Char	Tipe karakter.
String	type	String	Tipe string.
True	constant	Boolean	Bernilai 1 atau benar.
False	constant	Boolean	Bernilai 0 atau salah.
Writeln	procedure	void	Prosedur bawaan untuk menulis keluaran.
Readln	procedure	void	Prosedur bawaan untuk membaca masukan.

Reserved word diinisialisasi melalui fungsi `initReservedWords()`. Daftar reserved word disimpan pada vector `reservedWords`, kemudian setiap kata dimasukkan ke tab sebagai objek reserved. Reserved word tidak memiliki tipe data khusus sehingga atribut `type` dan `ref` bernilai 0. Setiap entri reserved word juga dihubungkan menggunakan atribut `link` untuk membentuk daftar identifier pada scope global.

Predefined identifier diinisialisasi melalui fungsi `initPredefinedIdentifiers()`. Identifier `Integer`, `Real`, `Boolean`, `Char`, dan `String` dimasukkan sebagai objek `type` dan menjadi representasi tipe dasar pada semantic analyzer. Identifier `True` dan `False` dimasukkan sebagai objek `constant` bertipe `Boolean`, dengan nilai disimpan pada atribut `adr`. Selain itu, `Writeln` dan `Readln` dimasukkan sebagai objek `procedure` agar dapat dikenali saat digunakan sebagai pemanggilan prosedur pada program sumber.

Operasi utama pada symbol table terdiri dari `insertTab()`, `insertBtab()`, `insertAtab()`, `lookup()`, `lookupInScope()`, `pushScope()`, dan `popScope()`. Fungsi `insertTab()` digunakan untuk menambahkan identifier baru ke tab pada scope aktif. Fungsi `insertBtab()` digunakan untuk menambahkan entri blok baru ke `btab`, sedangkan `insertAtab()` digunakan untuk menambahkan informasi tipe array ke `atab`. Fungsi `lookup()` digunakan untuk mencari identifier dari scope

terdalam menuju scope terluar. Pencarian dilakukan dengan memanggil `lookupInScope()` mulai dari `currentLevel()` hingga level global. Fungsi `lookupInScope()` mencari identifier hanya pada satu scope tertentu dengan menelusuri linked list.

Pengelolaan scope dilakukan menggunakan `display`. `display` menyimpan indeks blok aktif pada setiap lexical level. Ketika analyzer memasuki scope baru, `pushScope()` membuat entri baru pada `display` lalu menambahkan indeksnya ke `display`. Ketika analyzer keluar dari scope tersebut, `popScope()` menghapus indeks blok terakhir dari `display`. Dengan cara ini, lookup selalu dapat mengetahui scope aktif dan tetap dapat mencari identifier pada scope luar.

2.3 Semantic Analyzer dan Semantic Action

Komponen semantic analysis diimplementasikan pada `SemanticAnalyzer.hpp` dan `SemanticAnalyzer.cpp`. `SemanticAnalyzer` merupakan komponen yang melakukan traversal terhadap parse tree hasil parser. Pada implementasi ini, setiap fungsi visit berperan sebagai semantic action untuk produksi grammar tertentu. Semantic action tidak hanya membentuk node AST, tetapi juga melakukan registrasi identifier ke symbol table, lookup identifier, pengelolaan scope, inferensi tipe, validasi tipe, dan pengisian anotasi pada node AST.

Fungsi `analyze()` melakukan inisialisasi symbol table, mengosongkan daftar error dan warning, lalu memanggil fungsi `visit()` terhadap root parse tree. Fungsi `visit()` berperan sebagai dispatcher yang memanggil visitor sesuai jenis node parse tree, misalnya `visitProgram()`, `visitVarDeclaration()`, `visitAssignmentStatement()`, `visitExpression()`, dan sebagainya. Dengan demikian, pembentukan AST dan proses dekorasi AST dilakukan dalam satu traversal.

```
(inside) ./src/Semantic/SemanticAnalyzer.hpp
```

```
#ifndef __SEMANTIC_ANALYZER_HPP__
#define __SEMANTIC_ANALYZER_HPP__

#include "../Parser/Parser.hpp"
#include "AST.hpp"
#include "SymbolTable.hpp"
#include <string>
#include <vector>

class SemanticAnalyzer {
private:
    SymbolTable symTable;
    std::vector<std::string> errors;
    std::vector<std::string> warnings;

    int currentLine;

    ParseNode *findChild(ParseNode *node, const std::string &name) const;

    std::vector<ParseNode *> findAllChildren(ParseNode *node, const
std::string &name) const;

    ParseNode *getChild(ParseNode *node, int index) const;
```

```

std::string getToken(ParseNode *node) const;

bool hasChild(ParseNode *node, const std::string &name) const;

void addError(const std::string &msg);
void addWarning(const std::string &msg);

bool isCompatible(int typeCode1, int typeCode2) const;

bool isAssignmentCompatible(int targetType, int valueType) const;

bool isNumeric(int typeCode) const;

bool isBoolean(int typeCode) const;

bool isOrdinal(int typeCode) const;

int inferBinOpType(const std::string &op, int leftType, int rightType, int
line);

int inferUnaryOpType(const std::string &op, int operandType, int line);

std::string typeCodeToString(int typeCode) const;

ASTNode *visit(ParseNode *node);

ASTNode *visitProgram(ParseNode *node);
std::string visitProgramHeader(ParseNode *node);

ASTNode *visitDeclarationPart(ParseNode *node);
ASTNode *visitConstDeclaration(ParseNode *node);
ASTNode *visitConstant(ParseNode *node);
ASTNode *visitTypeDeclaration(ParseNode *node);
ASTNode *visitVarDeclaration(ParseNode *node);
std::vector<std::string> visitIdentifierList(ParseNode *node);

ASTNode *visitCompoundStatement(ParseNode *node);
ASTNode *visitStatementList(ParseNode *node);
ASTNode *visitAssignmentStatement(ParseNode *node);
ASTNode *visitProcedureCall(ParseNode *node);
ASTNode *visitIfStatement(ParseNode *node);
ASTNode *visitWhileStatement(ParseNode *node);
ASTNode *visitForStatement(ParseNode *node);
ASTNode *visitRepeatStatement(ParseNode *node);
ASTNode *visitCaseStatement(ParseNode *node);
ASTNode *visitCaseBlock(ParseNode *node);

ASTNode *visitExpression(ParseNode *node);
ASTNode *visitSimpleExpression(ParseNode *node);
ASTNode *visitTerm(ParseNode *node);
ASTNode *visitFactor(ParseNode *node);
ASTNode *visitVariable(ParseNode *node);
ASTNode *visitComponentVariable(ParseNode *node, ASTNode *base);

ASTNode *visitType(ParseNode *node, std::string &outTypeName);

```

```

ASTNode *visitArrayType(ParseNode *node, std::string &outTypeName);
ASTNode *visitRecordType(ParseNode *node, std::string &outTypeName);
ASTNode *visitRange(ParseNode *node, std::string &outTypeName);
ASTNode *visitEnumerated(ParseNode *node, std::string &outTypeName);

ASTNode *visitSubprogramDeclaration(ParseNode *node);
ASTNode *visitProcedureDeclaration(ParseNode *node);
ASTNode *visitFunctionDeclaration(ParseNode *node);
ASTNode *visitFormalParameterList(ParseNode *node, std::vector<VarDeclNode
*> &params);
ASTNode *visitParameterGroup(ParseNode *node, std::vector<VarDeclNode *>
&params);

std::string visitRelationalOperator(ParseNode *node);
std::string visitAdditiveOperator(ParseNode *node);
std::string visitMultiplicativeOperator(ParseNode *node);

public:
    SemanticAnalyzer();

    ASTNode *analyze(ParseNode *parseTree);

    bool hasErrors() const { return !errors.empty(); }

    const std::vector<std::string> &getErrors() const { return errors; }

    const std::vector<std::string> &getWarnings() const { return warnings; }

    const SymbolTable &getSymbolTable() const { return symTable; }

    void printResults(std::ostream &out, ASTNode *ast);
};

#endif

```

(inside) ./src/Semantic/SemanticAnalyzer.cpp

(source code terlalu panjang; bisa dibaca dari file)

Semantic action berfungsi untuk mengevaluasi nilai atribut dari masing-masing node. Secara umum, semantic action pada implementasi ini mencakup pembentukan node AST, penghilangan token sintaksis yang tidak diperlukan, penggabungan child node menjadi node induk, registrasi dan lookup identifier pada symbol table, pengelolaan scope, inferensi tipe, validasi kompatibilitas tipe, serta pengisian anotasi type, level, dan tabRef pada node AST. Rincian semantic action berdasarkan produksi grammar ditunjukkan pada tabel berikut.

Node Produksi	Semantic Action	Hasil Annotation
---------------	-----------------	------------------

<program>	Ambil nama program dari <program-header>, masukkan ke tab sebagai objek program, kunjungi declaration part dan compound statement.	Membentuk ProgramNode(name, decls, body) dengan tabRef ke entri program.
<var-declaration>	Ambil daftar identifier dan tipe, cek duplikasi pada scope aktif, masukkan tiap variabel ke tab.	Membentuk VarDeclNode; anotasi type, level, tabRef, dan adr.
<const-declaration>	Evaluasi nilai konstanta, tentukan tipenya, cek duplikasi, lalu masukkan konstanta ke tab.	Membentuk ConstDeclNode dengan type, level, dan tabRef.
<type-declaration>	Evaluasi definisi tipe, cek duplikasi nama tipe, lalu masukkan tipe baru ke tab.	Membentuk TypeDeclNode.
<assignment-statement>	Kunjungi target dan ekspresi kanan, lakukan lookup tipe target dan value, lalu cek isAssignmentCompatible.	Membentuk AssignNode(target, value); error jika tipe tidak kompatibel.
<expression>	Kunjungi operand kiri dan kanan, ambil operator relasional, lalu infer tipe hasil.	Membentuk BinOpNode(op, left, right) dengan tipe Boolean.
<simple-expression>	Evaluasi operasi +, -, or, termasuk unary + dan -.	Membentuk BinOpNode atau UnaryOpNode dengan anotasi type.
<term>	Evaluasi operasi *, /, div, mod, dan and.	Membentuk BinOpNode dengan tipe hasil operasi.
<factor>	Evaluasi literal, identifier, pemanggilan fungsi, ekspresi dalam kurung, atau operator not.	Membentuk node literal, VarNode, FuncCallNode, atau UnaryOpNode.
<variable>	Lookup identifier dari scope terdalam ke terluar.	Membentuk VarNode dengan type, level, dan tabRef.
<array-type>	Evaluasi tipe indeks dan tipe elemen, validasi indeks bukan	Membentuk ArrayTypeNode dengan referensi atab.

	Real, lalu simpan info array ke atab.	
<record-type>	Ambil daftar field dan tipe field.	Membentuk RecordTypeNode.
<if-statement>	Evaluasi kondisi, pastikan bertipe Boolean, lalu kunjungi statement then dan else.	Membentuk IfNode; error jika kondisi bukan Boolean.
<while-statement>	Evaluasi kondisi, pastikan bertipe Boolean, lalu kunjungi body.	Membentuk WhileNode; error jika kondisi bukan Boolean.
<for-statement>	Lookup variabel loop, evaluasi nilai awal dan akhir, baca arah to atau downto, lalu kunjungi body.	Membentuk ForNode; error jika variabel loop belum dideklarasikan.
<procedure/function-call>	Lookup nama prosedur/fungsi, validasi objeknya, lalu kunjungi argumen.	Membentuk ProcCallNode atau FuncCallNode dengan tabRef.
<program>	Ambil nama program dari <program-header>, masukkan ke tab sebagai objek program, kunjungi declaration part dan compound statement.	Membentuk ProgramNode(name, decls, body) dengan tabRef ke entri program.
<var-declaration>	Ambil daftar identifier dan tipe, cek duplikasi pada scope aktif, masukkan tiap variabel ke tab.	Membentuk VarDeclNode; anotasi type, level, tabRef, dan adr.
<const-declaration>	Evaluasi nilai konstanta, tentukan tipenya, cek duplikasi, lalu masukkan konstanta ke tab.	Membentuk ConstDeclNode dengan type, level, dan tabRef.
<type-declaration>	Evaluasi definisi tipe, cek duplikasi nama tipe, lalu masukkan tipe baru ke tab.	Membentuk TypeDeclNode.

2.4 Type Checking and Compatibility

Type checking dilakukan di dalam *SemanticAnalyzer.cpp* melalui fungsi `isCompatible()`, `isAssignmentCompatible()`, `inferBinOpType()`, dan `inferUnaryOpType()`. Fungsi-fungsi ini digunakan untuk menentukan apakah dua tipe dapat digunakan bersama dalam ekspresi, assignment, operator aritmetika, operator logika, operator relasional, serta kondisi pada struktur kontrol.

Fungsi `isCompatible()` digunakan untuk mengecek kompatibilitas umum antar tipe. Dua tipe dianggap kompatibel apabila memiliki type code yang sama, keduanya numerik, keduanya Boolean, keduanya Char, atau keduanya String. Fungsi `isAssignmentCompatible()` digunakan untuk mengecek assignment, dengan aturan tambahan bahwa nilai bertipe Integer dapat di-assign ke variabel bertipe Real.

Fungsi `inferBinOpType()` digunakan untuk menentukan tipe hasil operasi biner. Operator `+`, `-`, dan `*` hanya valid untuk operand numerik. Jika salah satu operand bertipe Real, hasilnya Real; jika keduanya Integer, hasilnya Integer. Operator `/` menghasilkan Real untuk operand numerik. Operator `div` dan `mod` hanya menerima Integer dan menghasilkan Integer. Operator `and` dan `or` hanya menerima Boolean dan menghasilkan Boolean. Operator relasional menghasilkan Boolean jika kedua operand kompatibel.

Fungsi `inferUnaryOpType()` digunakan untuk menentukan tipe hasil operasi unary. Operator unary `+` dan `-` hanya valid untuk operand numerik dan menghasilkan tipe yang sama dengan operand. Operator `not` hanya valid untuk operand Boolean dan menghasilkan Boolean.

Selain ekspresi, type checking juga dilakukan pada assignment statement dan struktur kontrol. Pada assignment, tipe target dan tipe ekspresi kanan harus assignment-compatible. Pada `if`, `while`, dan `repeat-until`, ekspresi kondisi harus bertipe Boolean.

Implementasi fungsi-fungsi pengecekan tipe tersebut ditunjukkan pada potongan kode berikut.

```
(inside) ./src/Semantic/SemanticAnalyzer.hpp
```

```
bool SemanticAnalyzer::isCompatible(int typeCode1, int typeCode2) const {
    if (typeCode1 == typeCode2)
        return true;

    if (isNumeric(typeCode1) && isNumeric(typeCode2))
        return true;

    if (isBoolean(typeCode1) && isBoolean(typeCode2))
        return true;

    if (typeCode1 == symTable.getTypeChar() && typeCode2 ==
        symTable.getTypeChar())
        return true;

    if (typeCode1 == symTable.getTypeString() && typeCode2 ==
        symTable.getTypeString())
        return true;

    return false;
}
```

```

}

bool SemanticAnalyzer::isAssignmentCompatible(int targetType, int valueType)
const {
    if (targetType == symTable.getTypeReal() && valueType ==
symTable.getTypeInteger())
        return true;

    if (isCompatible(targetType, valueType))
        return true;

    return false;
}

int SemanticAnalyzer::inferBinOpType(const std::string &op, int leftType, int
rightType, int /*line*/) {
    if (op == "+" || op == "-" || op == "*")
    {
        if (isNumeric(leftType) && isNumeric(rightType))
        {
            if (leftType == symTable.getTypeReal() || rightType ==
symTable.getTypeReal())
            {
                return symTable.getTypeReal();
            }
            return symTable.getTypeInteger();
        }
        addError("Operator '" + op + "' not defined for types '" +
            typeCodeToString(leftType) + "' and '" +
typeCodeToString(rightType) + "'");
        return 0;
    }

    if (op == "/")
    {
        if (isNumeric(leftType) && isNumeric(rightType))
        {
            return symTable.getTypeReal();
        }
        addError("Operator '/' not defined for types '" +
            typeCodeToString(leftType) + "' and '" +
typeCodeToString(rightType) + "'");
        return 0;
    }

    if (op == "div" || op == "mod")
    {
        if (leftType == symTable.getTypeInteger() && rightType ==
symTable.getTypeInteger())
        {
            return symTable.getTypeInteger();
        }
        addError("Operator '" + op + "' requires Integer operands, got '" +
            typeCodeToString(leftType) + "' and '" +
typeCodeToString(rightType) + "'");
    }
}

```

```

        return 0;
    }

    if (op == "and" || op == "or")
    {
        if (isBoolean(leftType) && isBoolean(rightType))
        {
            return symTable.getTypeBoolean();
        }
        addError("Operator '" + op + "' requires Boolean operands, got '" +
            typeCodeToString(leftType) + "' and '" +
typeCodeToString(rightType) + "'");
        return 0;
    }

    if (op == "=" || op == "<>" || op == "<" || op == ">" || op == "<=" || op
== ">=")
    {
        if (isCompatible(leftType, rightType))
        {
            return symTable.getTypeBoolean();
        }
        addError("Cannot compare types '" + typeCodeToString(leftType) +
            "' and '" + typeCodeToString(rightType) + "' with operator '"
+ op + "'");
        return 0;
    }

    return 0;
}

int SemanticAnalyzer::inferUnaryOpType(const std::string &op, int operandType,
int /*line*/) {
    if (op == "+" || op == "-")
    {
        if (isNumeric(operandType))
        {
            return operandType;
        }
        addError("Unary operator '" + op + "' requires numeric operand, got '"
+
            typeCodeToString(operandType) + "'");
        return 0;
    }
    if (op == "not")
    {
        if (isBoolean(operandType))
        {
            return symTable.getTypeBoolean();
        }
        addError("Operator 'not' requires Boolean operand, got '" +
            typeCodeToString(operandType) + "'");
        return 0;
    }
    return 0;
}

```

Type compatibility rule yang digunakan adalah sebagai berikut.

Kategori	Aturan
Tipe sama	Dua tipe kompatibel jika keduanya memiliki type code yang sama.
Numerik	Integer dan Real dianggap kompatibel untuk operasi numerik.
Boolean	Operasi logika hanya valid jika kedua operand bertipe Boolean.
Char	Char hanya kompatibel dengan Char.
String	String hanya kompatibel dengan String.
Assignment tipe sama	Assignment valid jika tipe target dan tipe nilai kompatibel.
Assignment Integer ke Real	Assignment dari Integer ke Real diperbolehkan.
Assignment tidak kompatibel	Assignment menghasilkan error jika tipe target dan tipe nilai tidak kompatibel.
Operator +, -, *	Operand harus numerik. Jika salah satu operand Real, hasilnya Real; jika keduanya Integer, hasilnya Integer.
Operator /	Operand harus numerik dan hasil selalu Real.
Operator div, mod	Kedua operand harus Integer dan hasilnya Integer.
Operator and, or	Kedua operand harus Boolean dan hasilnya Boolean.
Operator not	Operand harus Boolean dan hasilnya Boolean.
Operator unary +, -	Operand harus numerik dan hasilnya mengikuti tipe operand.
Operator relasional =, <>, <, >, <=, >=	Kedua operand harus kompatibel dan hasilnya Boolean.
Kondisi if	Ekspresi kondisi harus bertipe Boolean.
Kondisi while	Ekspresi kondisi harus bertipe Boolean.
Kondisi repeat-until	Ekspresi kondisi harus bertipe Boolean.
Indeks array	Tipe indeks array tidak boleh Real.

Subrange	Batas bawah dan batas atas harus bertipe valid, tidak boleh Real, dan batas bawah tidak boleh lebih besar dari batas atas.
----------	--

2.5 Alur Program

Berikut adalah kode main.cpp dari program:

```
./src/main.cpp

#include "Lexer/Lexer.hpp"
#include "Parser/Parser.hpp"
#include "Semantic/SemanticAnalyzer.hpp"
#include <string>
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[]) {
    if (argc < 2) {
        cerr << "Usage: ./bin/arion-compiler [--parse|--semantic]
<input-file.txt>" << endl;
        cerr << "  Default: lexer only" << endl;
        cerr << "  --parse: run lexer + parser and output parse tree" << endl;
        cerr << "  --semantic: run parser + semantic analysis and output
decorated AST + symbol tables" << endl;
        cerr << "  Example: ./bin/arion-compiler test.txt" << endl;
        cerr << "  Example: ./bin/arion-compiler --parse test.txt" << endl;
        cerr << "  Example: ./bin/arion-compiler --semantic test.txt" << endl;
        return 1;
    }

    bool parseMode = false;
    bool semanticMode = false;
    string filename;

    if (argc >= 3) {
        if (string(argv[1]) == "--parse") {
            parseMode = true;
            filename = argv[2];
        }
        else if (string(argv[1]) == "--semantic") {
            semanticMode = true;
            filename = argv[2];
        }
        else {
            filename = argv[1];
        }
    }
    else {
        filename = argv[1];
    }
}
```

```

try {
    if (parseMode) {
        // Parser mode
        Lexer lexer(filename);
        Parser parser(lexer);
        ParseNode *tree = parser.parseProgram();

        // Print tree to stdout
        tree->print(cout);

        // Save tree to output file
        string outfile = filename;
        size_t lastDot = outfile.rfind('.');
        if (lastDot != string::npos) {
            outfile = outfile.substr(0, lastDot) + "_parse.txt";
        }
        else {
            outfile = outfile + "_parse.txt";
        }
        ofstream out(outfile);
        tree->print(out);
        out.close();
        cerr << "Parse tree saved to: " << outfile << endl;

        delete tree;
    }
    else if (semanticMode) {
        // Semantic Analysis mode
        Lexer lexer(filename);
        Parser parser(lexer);
        ParseNode *tree = parser.parseProgram();

        // Run semantic analysis
        SemanticAnalyzer analyzer;
        ASTNode *ast = analyzer.analyze(tree);

        // Print results to stdout
        analyzer.printResults(cout, ast);

        // Save results to output file
        string outfile = filename;
        size_t lastDot = outfile.rfind('.');
        if (lastDot != string::npos) {
            outfile = outfile.substr(0, lastDot) + "_semantic.txt";
        }
        else {
            outfile = outfile + "_semantic.txt";
        }
        ofstream out(outfile);
        analyzer.printResults(out, ast);
        out.close();
        cerr << "Semantic analysis saved to: " << outfile << endl;

        delete ast;
        delete tree;
    }
}

```

```

    }
    else {
        // Lexer mode (default)
        Lexer lexer(filename);
        Token token = lexer.getNextToken();
        while (token.type != TokenType::END_OF_FILE) {
            string typeName = lexer.tokenTypeToString(token.type);

            if (token.type == TokenType::IDENT || token.type ==
TokenType::INTCON || token.type == TokenType::REALCON) {
                cout << typeName << " (" << token.value << ")" << endl;
            }
            else if (token.type == TokenType::STRING || token.type ==
TokenType::CHARCON) {
                cout << typeName << " ('" << token.value << "')" << endl;
            }
            else if (token.type == TokenType::UNKNOWN) {
                cout << typeName << " (" << token.value << ")" << endl;
            }
            else {
                cout << typeName << endl;
            }
            token = lexer.getNextToken();
        }
    }
}
catch (const exception &e) {
    cerr << "Error: " << e.what() << endl;
    return 1;
}
return 0;
}

```

Pada mode --semantic, program pertama-tama membuat objek Lexer dengan file input yang diberikan pengguna. Lexer bertugas membaca source code dan menghasilkan token stream. Setelah itu, objek Parser dibuat dengan menerima Lexer sebagai masukan. Parser kemudian menjalankan parseProgram() untuk membentuk parse tree dari token-token tersebut.

Parse tree yang dihasilkan parser diberikan kepada objek SemanticAnalyzer melalui fungsi analyze(). Di dalam fungsi ini, semantic analyzer melakukan inisialisasi symbol table, traversal terhadap parse tree, pembentukan Decorated AST, pengisian symbol table, serta pengecekan semantik seperti deklarasi identifier, scope, dan kompatibilitas tipe.

Setelah proses semantic analysis selesai, hasilnya dicetak menggunakan printResults(). Fungsi ini menampilkan Decorated AST, tabel tab, btab, atab, daftar error, dan daftar warning. Hasil yang sama juga disimpan ke file output dengan nama yang sama seperti file input, tetapi menggunakan akhiran _semantic.txt.

BAB 3

Pengujian

input-1.txt

=== Decorated AST ===

```

ProgramNode(name: 'ValidBasic') -> tab_index:41
├── DeclarationListNode
│   └── DeclarationListNode
│       ├── VarDecl('a') -> tab_index:42, type:integer, lev:0
│       └── VarDecl('b') -> tab_index:43, type:integer, lev:0
└── BlockNode -> block_index:0
    ├── DeclarationListNode
    │   ├── Assign -> type:void
    │   │   ├── 'a' -> tab_index:42, type:Integer
    │   │   └── 5 -> type:Integer
    │   ├── Assign -> type:void
    │   │   ├── 'b' -> tab_index:43, type:Integer
    │   │   └── BinOp '+' -> type:Integer
    │   │       ├── 'a' -> tab_index:42, type:Integer
    │   │       └── 10 -> type:Integer
    └── ProcCall(writeln) -> tab_index:39
        └── 'b' -> tab_index:43, type:Integer
    
```

=== tab (Identifier Table) ===

idx	id	obj	type	ref	nrm	lev	adr	link
0	and	reserved	0	0	1	0	0	0
1	array	reserved	0	0	1	0	0	0
2	begin	reserved	0	0	1	0	0	1
3	case	reserved	0	0	1	0	0	2
4	const	reserved	0	0	1	0	0	3
5	div	reserved	0	0	1	0	0	4
6	downto	reserved	0	0	1	0	0	5
7	do	reserved	0	0	1	0	0	6
8	else	reserved	0	0	1	0	0	7
9	end	reserved	0	0	1	0	0	8
10	for	reserved	0	0	1	0	0	9
11	function	reserved	0	0	1	0	0	10
12	if	reserved	0	0	1	0	0	11
13	mod	reserved	0	0	1	0	0	12
14	not	reserved	0	0	1	0	0	13
15	of	reserved	0	0	1	0	0	14
16	or	reserved	0	0	1	0	0	15
17	procedure	reserved	0	0	1	0	0	16
18	program	reserved	0	0	1	0	0	17
19	record	reserved	0	0	1	0	0	18
20	repeat	reserved	0	0	1	0	0	19
21	integer	reserved	0	0	1	0	0	20
22	real	reserved	0	0	1	0	0	21
23	boolean	reserved	0	0	1	0	0	22
24	char	reserved	0	0	1	0	0	23
25	string	reserved	0	0	1	0	0	24
26	then	reserved	0	0	1	0	0	25
27	to	reserved	0	0	1	0	0	26

	<pre> 28 type reserved 0 0 1 0 0 27 29 until reserved 0 0 1 0 0 28 30 var reserved 0 0 1 0 0 29 31 while reserved 0 0 1 0 0 30 32 Integer type 32 0 1 0 0 31 33 Real type 33 0 1 0 0 32 34 Boolean type 34 0 1 0 0 33 35 Char type 35 0 1 0 0 34 36 String type 36 0 1 0 0 35 37 True constant 34 0 1 0 1 36 38 False constant 34 0 1 0 0 37 39 Writeln procedure 0 0 1 0 0 38 40 Readln procedure 0 0 1 0 0 39 41 ValidBasic program 0 0 1 0 0 40 42 a variable 32 0 1 0 0 41 43 b variable 32 0 1 0 1 42 === btab (Block Table) === idx last lpar psze vsze ----- 0 43 0 0 2 === atab (Array Table) === (empty) === Errors === No errors found. === Warnings === No warnings.</pre>
input-2.txt	<pre> === Decorated AST === ProgramNode(name: 'ConstExpression') -> tab_index:41 ├─ DeclarationListNode │ └─ DeclarationListNode │ ├── ConstDecl('base') -> tab_index:42, type:Integer, lev:0 │ │ └─ 10 -> type:Integer │ ├── ConstDecl('inc') -> tab_index:43, type:Integer, lev:0 │ │ └─ 5 -> type:Integer │ └─ DeclarationListNode │ └─ VarDecl('result') -> tab_index:44, type:integer, lev:0 └─ BlockNode -> block_index:0 └─ DeclarationListNode ├── Assign -> type:void │ ├── 'result' -> tab_index:44, type:Integer │ └─ BinOp '+' -> type:Integer │ ├── 10 -> type:Integer │ └─ BinOp '*' -> type:Integer │ ├── 5 -> type:Integer │ └─ 2 -> type:Integer └─ ProcCall(writeln) -> tab_index:39 └─ 'result' -> tab_index:44, type:Integer === tab (Identifier Table) === idx id obj type ref nrm lev adr link -----</pre>

0	and	reserved	0	0	1	0	0	0
1	array	reserved	0	0	1	0	0	0
2	begin	reserved	0	0	1	0	0	1
3	case	reserved	0	0	1	0	0	2
4	const	reserved	0	0	1	0	0	3
5	div	reserved	0	0	1	0	0	4
6	downto	reserved	0	0	1	0	0	5
7	do	reserved	0	0	1	0	0	6
8	else	reserved	0	0	1	0	0	7
9	end	reserved	0	0	1	0	0	8
10	for	reserved	0	0	1	0	0	9
11	function	reserved	0	0	1	0	0	10
12	if	reserved	0	0	1	0	0	11
13	mod	reserved	0	0	1	0	0	12
14	not	reserved	0	0	1	0	0	13
15	of	reserved	0	0	1	0	0	14
16	or	reserved	0	0	1	0	0	15
17	procedure	reserved	0	0	1	0	0	16
18	program	reserved	0	0	1	0	0	17
19	record	reserved	0	0	1	0	0	18
20	repeat	reserved	0	0	1	0	0	19
21	integer	reserved	0	0	1	0	0	20
22	real	reserved	0	0	1	0	0	21
23	boolean	reserved	0	0	1	0	0	22
24	char	reserved	0	0	1	0	0	23
25	string	reserved	0	0	1	0	0	24
26	then	reserved	0	0	1	0	0	25
27	to	reserved	0	0	1	0	0	26
28	type	reserved	0	0	1	0	0	27
29	until	reserved	0	0	1	0	0	28
30	var	reserved	0	0	1	0	0	29
31	while	reserved	0	0	1	0	0	30
32	Integer	type	32	0	1	0	0	31
33	Real	type	33	0	1	0	0	32
34	Boolean	type	34	0	1	0	0	33
35	Char	type	35	0	1	0	0	34
36	String	type	36	0	1	0	0	35
37	True	constant	34	0	1	0	1	36
38	False	constant	34	0	1	0	0	37
39	Writeln	procedure	0	0	1	0	0	38
40	Readln	procedure	0	0	1	0	0	39
41	ConstExpression	program	0	0	1	0	0	40
42	base	constant	32	0	1	0	10	41
43	inc	constant	32	0	1	0	5	42
44	result	variable	32	0	1	0	0	43

=== btab (Block Table) ===

idx	last	lpar	psze	vsze
0	44	0	0	1

=== atab (Array Table) ===

(empty)

=== Errors ===

No errors found.

	<pre>=== Warnings === No warnings.</pre>																																																																																																																																																																																																																																																																																																																																																																								
input-3.txt	<pre>=== Decorated AST === ProgramNode(name: 'DuplicateIdentifier') -> tab_index:41 ├─ DeclarationListNode │ └─ DeclarationListNode │ └─ VarDecl('x') -> tab_index:42, type:integer, lev:0 └─ BlockNode -> block_index:0 └─ DeclarationListNode └─ Assign -> type:void └─ 'x' -> tab_index:42, type:Integer └─ 5 -> type:Integer</pre> <pre>=== tab (Identifier Table) ===</pre> <table><tr><th>idx</th><th>id</th><th>obj</th><th>type</th><th>ref</th><th>nrm</th><th>lev</th><th>adr</th><th>link</th></tr><tr><td colspan="9">-----</td></tr><tr><td>0</td><td>and</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>array</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td></tr><tr><td>2</td><td>begin</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>3</td><td>case</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>2</td></tr><tr><td>4</td><td>const</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>3</td></tr><tr><td>5</td><td>div</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>4</td></tr><tr><td>6</td><td>downto</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>5</td></tr><tr><td>7</td><td>do</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>6</td></tr><tr><td>8</td><td>else</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>7</td></tr><tr><td>9</td><td>end</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>8</td></tr><tr><td>10</td><td>for</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>9</td></tr><tr><td>11</td><td>function</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>10</td></tr><tr><td>12</td><td>if</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>11</td></tr><tr><td>13</td><td>mod</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>12</td></tr><tr><td>14</td><td>not</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>13</td></tr><tr><td>15</td><td>of</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>14</td></tr><tr><td>16</td><td>or</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>15</td></tr><tr><td>17</td><td>procedure</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>16</td></tr><tr><td>18</td><td>program</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>17</td></tr><tr><td>19</td><td>record</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>18</td></tr><tr><td>20</td><td>repeat</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>19</td></tr><tr><td>21</td><td>integer</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>20</td></tr><tr><td>22</td><td>real</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>21</td></tr><tr><td>23</td><td>boolean</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>22</td></tr><tr><td>24</td><td>char</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>23</td></tr><tr><td>25</td><td>string</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>24</td></tr><tr><td>26</td><td>then</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>25</td></tr><tr><td>27</td><td>to</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>26</td></tr><tr><td>28</td><td>type</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>27</td></tr><tr><td>29</td><td>until</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>28</td></tr><tr><td>30</td><td>var</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>29</td></tr><tr><td>31</td><td>while</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>30</td></tr><tr><td>32</td><td>Integer</td><td>type</td><td>32</td><td>0</td><td>1</td><td>0</td><td>0</td><td>31</td></tr><tr><td>33</td><td>Real</td><td>type</td><td>33</td><td>0</td><td>1</td><td>0</td><td>0</td><td>32</td></tr><tr><td>34</td><td>Boolean</td><td>type</td><td>34</td><td>0</td><td>1</td><td>0</td><td>0</td><td>33</td></tr><tr><td>35</td><td>Char</td><td>type</td><td>35</td><td>0</td><td>1</td><td>0</td><td>0</td><td>34</td></tr><tr><td>36</td><td>String</td><td>type</td><td>36</td><td>0</td><td>1</td><td>0</td><td>0</td><td>35</td></tr><tr><td>37</td><td>True</td><td>constant</td><td>34</td><td>0</td><td>1</td><td>0</td><td>1</td><td>36</td></tr></table>	idx	id	obj	type	ref	nrm	lev	adr	link	-----									0	and	reserved	0	0	1	0	0	0	1	array	reserved	0	0	1	0	0	0	2	begin	reserved	0	0	1	0	0	1	3	case	reserved	0	0	1	0	0	2	4	const	reserved	0	0	1	0	0	3	5	div	reserved	0	0	1	0	0	4	6	downto	reserved	0	0	1	0	0	5	7	do	reserved	0	0	1	0	0	6	8	else	reserved	0	0	1	0	0	7	9	end	reserved	0	0	1	0	0	8	10	for	reserved	0	0	1	0	0	9	11	function	reserved	0	0	1	0	0	10	12	if	reserved	0	0	1	0	0	11	13	mod	reserved	0	0	1	0	0	12	14	not	reserved	0	0	1	0	0	13	15	of	reserved	0	0	1	0	0	14	16	or	reserved	0	0	1	0	0	15	17	procedure	reserved	0	0	1	0	0	16	18	program	reserved	0	0	1	0	0	17	19	record	reserved	0	0	1	0	0	18	20	repeat	reserved	0	0	1	0	0	19	21	integer	reserved	0	0	1	0	0	20	22	real	reserved	0	0	1	0	0	21	23	boolean	reserved	0	0	1	0	0	22	24	char	reserved	0	0	1	0	0	23	25	string	reserved	0	0	1	0	0	24	26	then	reserved	0	0	1	0	0	25	27	to	reserved	0	0	1	0	0	26	28	type	reserved	0	0	1	0	0	27	29	until	reserved	0	0	1	0	0	28	30	var	reserved	0	0	1	0	0	29	31	while	reserved	0	0	1	0	0	30	32	Integer	type	32	0	1	0	0	31	33	Real	type	33	0	1	0	0	32	34	Boolean	type	34	0	1	0	0	33	35	Char	type	35	0	1	0	0	34	36	String	type	36	0	1	0	0	35	37	True	constant	34	0	1	0	1	36
idx	id	obj	type	ref	nrm	lev	adr	link																																																																																																																																																																																																																																																																																																																																																																	

0	and	reserved	0	0	1	0	0	0																																																																																																																																																																																																																																																																																																																																																																	
1	array	reserved	0	0	1	0	0	0																																																																																																																																																																																																																																																																																																																																																																	
2	begin	reserved	0	0	1	0	0	1																																																																																																																																																																																																																																																																																																																																																																	
3	case	reserved	0	0	1	0	0	2																																																																																																																																																																																																																																																																																																																																																																	
4	const	reserved	0	0	1	0	0	3																																																																																																																																																																																																																																																																																																																																																																	
5	div	reserved	0	0	1	0	0	4																																																																																																																																																																																																																																																																																																																																																																	
6	downto	reserved	0	0	1	0	0	5																																																																																																																																																																																																																																																																																																																																																																	
7	do	reserved	0	0	1	0	0	6																																																																																																																																																																																																																																																																																																																																																																	
8	else	reserved	0	0	1	0	0	7																																																																																																																																																																																																																																																																																																																																																																	
9	end	reserved	0	0	1	0	0	8																																																																																																																																																																																																																																																																																																																																																																	
10	for	reserved	0	0	1	0	0	9																																																																																																																																																																																																																																																																																																																																																																	
11	function	reserved	0	0	1	0	0	10																																																																																																																																																																																																																																																																																																																																																																	
12	if	reserved	0	0	1	0	0	11																																																																																																																																																																																																																																																																																																																																																																	
13	mod	reserved	0	0	1	0	0	12																																																																																																																																																																																																																																																																																																																																																																	
14	not	reserved	0	0	1	0	0	13																																																																																																																																																																																																																																																																																																																																																																	
15	of	reserved	0	0	1	0	0	14																																																																																																																																																																																																																																																																																																																																																																	
16	or	reserved	0	0	1	0	0	15																																																																																																																																																																																																																																																																																																																																																																	
17	procedure	reserved	0	0	1	0	0	16																																																																																																																																																																																																																																																																																																																																																																	
18	program	reserved	0	0	1	0	0	17																																																																																																																																																																																																																																																																																																																																																																	
19	record	reserved	0	0	1	0	0	18																																																																																																																																																																																																																																																																																																																																																																	
20	repeat	reserved	0	0	1	0	0	19																																																																																																																																																																																																																																																																																																																																																																	
21	integer	reserved	0	0	1	0	0	20																																																																																																																																																																																																																																																																																																																																																																	
22	real	reserved	0	0	1	0	0	21																																																																																																																																																																																																																																																																																																																																																																	
23	boolean	reserved	0	0	1	0	0	22																																																																																																																																																																																																																																																																																																																																																																	
24	char	reserved	0	0	1	0	0	23																																																																																																																																																																																																																																																																																																																																																																	
25	string	reserved	0	0	1	0	0	24																																																																																																																																																																																																																																																																																																																																																																	
26	then	reserved	0	0	1	0	0	25																																																																																																																																																																																																																																																																																																																																																																	
27	to	reserved	0	0	1	0	0	26																																																																																																																																																																																																																																																																																																																																																																	
28	type	reserved	0	0	1	0	0	27																																																																																																																																																																																																																																																																																																																																																																	
29	until	reserved	0	0	1	0	0	28																																																																																																																																																																																																																																																																																																																																																																	
30	var	reserved	0	0	1	0	0	29																																																																																																																																																																																																																																																																																																																																																																	
31	while	reserved	0	0	1	0	0	30																																																																																																																																																																																																																																																																																																																																																																	
32	Integer	type	32	0	1	0	0	31																																																																																																																																																																																																																																																																																																																																																																	
33	Real	type	33	0	1	0	0	32																																																																																																																																																																																																																																																																																																																																																																	
34	Boolean	type	34	0	1	0	0	33																																																																																																																																																																																																																																																																																																																																																																	
35	Char	type	35	0	1	0	0	34																																																																																																																																																																																																																																																																																																																																																																	
36	String	type	36	0	1	0	0	35																																																																																																																																																																																																																																																																																																																																																																	
37	True	constant	34	0	1	0	1	36																																																																																																																																																																																																																																																																																																																																																																	

	<pre> 38 False constant 34 0 1 0 0 37 39 Writeln procedure 0 0 1 0 0 38 40 Readln procedure 0 0 1 0 0 39 41 DuplicateIdentifierprogram 0 0 1 0 0 40 42 x variable 32 0 1 0 0 41 === btab (Block Table) === idx last lpar psze vsze ----- 0 42 0 0 1 === atab (Array Table) === (empty) === Errors === Error: Duplicate identifier 'x' in current scope === Warnings === No warnings.</pre>
input-4.txt	<pre> === Decorated AST === ProgramNode(name: 'UndeclaredIdentifier') -> tab_index:41 ├─ DeclarationListNode │ └─ DeclarationListNode │ └─ VarDecl('a') -> tab_index:42, type:integer, lev:0 └─ BlockNode -> block_index:0 └─ DeclarationListNode ├── Assign -> type:void │ ├── 'a' -> tab_index:42, type:Integer │ └─ 1 -> type:Integer └─ Assign -> type:void ├── 'b' -> tab_index:0, type:? └─ BinOp '+' -> type:Integer ├── 'a' -> tab_index:42, type:Integer └─ 2 -> type:Integer === tab (Identifier Table) === idx id obj type ref nrm lev adr link ----- 0 and reserved 0 0 1 0 0 0 1 array reserved 0 0 1 0 0 0 2 begin reserved 0 0 1 0 0 1 3 case reserved 0 0 1 0 0 2 4 const reserved 0 0 1 0 0 3 5 div reserved 0 0 1 0 0 4 6 downto reserved 0 0 1 0 0 5 7 do reserved 0 0 1 0 0 6 8 else reserved 0 0 1 0 0 7 9 end reserved 0 0 1 0 0 8 10 for reserved 0 0 1 0 0 9 11 function reserved 0 0 1 0 0 10 12 if reserved 0 0 1 0 0 11 13 mod reserved 0 0 1 0 0 12 14 not reserved 0 0 1 0 0 13 15 of reserved 0 0 1 0 0 14 16 or reserved 0 0 1 0 0 15</pre>

	<pre> 17 procedure reserved 0 0 1 0 0 16 18 program reserved 0 0 1 0 0 17 19 record reserved 0 0 1 0 0 18 20 repeat reserved 0 0 1 0 0 19 21 integer reserved 0 0 1 0 0 20 22 real reserved 0 0 1 0 0 21 23 boolean reserved 0 0 1 0 0 22 24 char reserved 0 0 1 0 0 23 25 string reserved 0 0 1 0 0 24 26 then reserved 0 0 1 0 0 25 27 to reserved 0 0 1 0 0 26 28 type reserved 0 0 1 0 0 27 29 until reserved 0 0 1 0 0 28 30 var reserved 0 0 1 0 0 29 31 while reserved 0 0 1 0 0 30 32 Integer type 32 0 1 0 0 31 33 Real type 33 0 1 0 0 32 34 Boolean type 34 0 1 0 0 33 35 Char type 35 0 1 0 0 34 36 String type 36 0 1 0 0 35 37 True constant 34 0 1 0 1 36 38 False constant 34 0 1 0 0 37 39 Writeln procedure 0 0 1 0 0 38 40 Readln procedure 0 0 1 0 0 39 41 UndeclaredIdentifierprogram 0 0 1 0 0 40 42 a variable 32 0 1 0 0 41 === btab (Block Table) === idx last lpar psze vsze ----- 0 42 0 0 1 === atab (Array Table) === (empty) === Errors === Error: Undeclared identifier 'b' === Warnings === No warnings. </pre>
input-5.txt	<pre> === Decorated AST === ProgramNode(name: 'InvalidAssignment') -> tab_index:41 ├─ DeclarationListNode │ └─ DeclarationListNode │ ├── VarDecl('n') -> tab_index:42, type:integer, lev:0 │ └─ VarDecl('s') -> tab_index:43, type:string, lev:0 └─ BlockNode -> block_index:0 └─ DeclarationListNode ├── Assign -> type:void │ ├── 's' -> tab_index:43, type:String │ └─ 'hello' -> type:String └─ Assign -> type:void ├── 'n' -> tab_index:42, type:Integer └─ 's' -> tab_index:43, type:String </pre>

```

=== tab (Identifier Table) ===
idx  id      obj      type  ref  nrm  lev  adr  link
-----
0    and      reserved  0    0    1    0    0    0
1    array     reserved  0    0    1    0    0    0
2    begin     reserved  0    0    1    0    0    1
3    case      reserved  0    0    1    0    0    2
4    const     reserved  0    0    1    0    0    3
5    div       reserved  0    0    1    0    0    4
6    downto    reserved  0    0    1    0    0    5
7    do        reserved  0    0    1    0    0    6
8    else      reserved  0    0    1    0    0    7
9    end       reserved  0    0    1    0    0    8
10   for       reserved  0    0    1    0    0    9
11   function   reserved  0    0    1    0    0    10
12   if        reserved  0    0    1    0    0    11
13   mod       reserved  0    0    1    0    0    12
14   not       reserved  0    0    1    0    0    13
15   of        reserved  0    0    1    0    0    14
16   or        reserved  0    0    1    0    0    15
17   procedure  reserved  0    0    1    0    0    16
18   program   reserved  0    0    1    0    0    17
19   record    reserved  0    0    1    0    0    18
20   repeat    reserved  0    0    1    0    0    19
21   integer   reserved  0    0    1    0    0    20
22   real      reserved  0    0    1    0    0    21
23   boolean   reserved  0    0    1    0    0    22
24   char      reserved  0    0    1    0    0    23
25   string    reserved  0    0    1    0    0    24
26   then      reserved  0    0    1    0    0    25
27   to        reserved  0    0    1    0    0    26
28   type      reserved  0    0    1    0    0    27
29   until     reserved  0    0    1    0    0    28
30   var       reserved  0    0    1    0    0    29
31   while     reserved  0    0    1    0    0    30
32   Integer   type      32    0    1    0    0    31
33   Real      type      33    0    1    0    0    32
34   Boolean   type      34    0    1    0    0    33
35   Char      type      35    0    1    0    0    34
36   String    type      36    0    1    0    0    35
37   True      constant  34    0    1    0    1    36
38   False     constant  34    0    1    0    0    37
39   Writeln   procedure  0    0    1    0    0    38
40   Readln    procedure  0    0    1    0    0    39
41   InvalidAssignmentprogram  0    0    1    0    0    40
42   n         variable  32    0    1    0    0    41
43   s         variable  36    0    1    0    1    42

=== btab (Block Table) ===
idx  last  lpar  psze  vsze
-----
0    43    0     0     2

=== atab (Array Table) ===
(empty)

```

	<pre>=== Errors === Error: Assignment incompatible: cannot assign value of type 'String' to variable of type 'Integer' === Warnings === No warnings.</pre>																																																																																																																																																																																																															
input-6.txt	<pre>=== Decorated AST === ProgramNode(name: 'BooleanExpression') -> tab_index:41 ├── DeclarationListNode │ └── DeclarationListNode │ ├── VarDecl('a') -> tab_index:42, type:integer, lev:0 │ ├── VarDecl('b') -> tab_index:43, type:integer, lev:0 │ └── VarDecl('ok') -> tab_index:44, type:boolean, lev:0 └── BlockNode -> block_index:0 ├── DeclarationListNode │ ├── Assign -> type:void │ │ ├── 'a' -> tab_index:42, type:Integer │ │ └── 10 -> type:Integer │ ├── Assign -> type:void │ │ ├── 'b' -> tab_index:43, type:Integer │ │ └── 20 -> type:Integer │ └── Assign -> type:void │ ├── 'ok' -> tab_index:44, type:Boolean │ └── BinOp 'and' -> type:Boolean │ ├── BinOp '<' -> type:Boolean │ │ ├── 'a' -> tab_index:42, type:Integer │ │ └── 'b' -> tab_index:43, type:Integer │ └── true -> type:Boolean └── ProcCall(writeln) -> tab_index:39 └── 'ok' -> tab_index:44, type:Boolean</pre> <pre>=== tab (Identifier Table) ===</pre> <table><tr><th>idx</th><th>id</th><th>obj</th><th>type</th><th>ref</th><th>nrm</th><th>lev</th><th>adr</th><th>link</th></tr><tr><td colspan="9">-----</td></tr><tr><td>0</td><td>and</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>array</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td></tr><tr><td>2</td><td>begin</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>3</td><td>case</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>2</td></tr><tr><td>4</td><td>const</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>3</td></tr><tr><td>5</td><td>div</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>4</td></tr><tr><td>6</td><td>downto</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>5</td></tr><tr><td>7</td><td>do</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>6</td></tr><tr><td>8</td><td>else</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>7</td></tr><tr><td>9</td><td>end</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>8</td></tr><tr><td>10</td><td>for</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>9</td></tr><tr><td>11</td><td>function</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>10</td></tr><tr><td>12</td><td>if</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>11</td></tr><tr><td>13</td><td>mod</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>12</td></tr><tr><td>14</td><td>not</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>13</td></tr><tr><td>15</td><td>of</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>14</td></tr><tr><td>16</td><td>or</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>15</td></tr><tr><td>17</td><td>procedure</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>16</td></tr><tr><td>18</td><td>program</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>17</td></tr><tr><td>19</td><td>record</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>18</td></tr><tr><td>20</td><td>repeat</td><td>reserved</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>19</td></tr></table>	idx	id	obj	type	ref	nrm	lev	adr	link	-----									0	and	reserved	0	0	1	0	0	0	1	array	reserved	0	0	1	0	0	0	2	begin	reserved	0	0	1	0	0	1	3	case	reserved	0	0	1	0	0	2	4	const	reserved	0	0	1	0	0	3	5	div	reserved	0	0	1	0	0	4	6	downto	reserved	0	0	1	0	0	5	7	do	reserved	0	0	1	0	0	6	8	else	reserved	0	0	1	0	0	7	9	end	reserved	0	0	1	0	0	8	10	for	reserved	0	0	1	0	0	9	11	function	reserved	0	0	1	0	0	10	12	if	reserved	0	0	1	0	0	11	13	mod	reserved	0	0	1	0	0	12	14	not	reserved	0	0	1	0	0	13	15	of	reserved	0	0	1	0	0	14	16	or	reserved	0	0	1	0	0	15	17	procedure	reserved	0	0	1	0	0	16	18	program	reserved	0	0	1	0	0	17	19	record	reserved	0	0	1	0	0	18	20	repeat	reserved	0	0	1	0	0	19
idx	id	obj	type	ref	nrm	lev	adr	link																																																																																																																																																																																																								

0	and	reserved	0	0	1	0	0	0																																																																																																																																																																																																								
1	array	reserved	0	0	1	0	0	0																																																																																																																																																																																																								
2	begin	reserved	0	0	1	0	0	1																																																																																																																																																																																																								
3	case	reserved	0	0	1	0	0	2																																																																																																																																																																																																								
4	const	reserved	0	0	1	0	0	3																																																																																																																																																																																																								
5	div	reserved	0	0	1	0	0	4																																																																																																																																																																																																								
6	downto	reserved	0	0	1	0	0	5																																																																																																																																																																																																								
7	do	reserved	0	0	1	0	0	6																																																																																																																																																																																																								
8	else	reserved	0	0	1	0	0	7																																																																																																																																																																																																								
9	end	reserved	0	0	1	0	0	8																																																																																																																																																																																																								
10	for	reserved	0	0	1	0	0	9																																																																																																																																																																																																								
11	function	reserved	0	0	1	0	0	10																																																																																																																																																																																																								
12	if	reserved	0	0	1	0	0	11																																																																																																																																																																																																								
13	mod	reserved	0	0	1	0	0	12																																																																																																																																																																																																								
14	not	reserved	0	0	1	0	0	13																																																																																																																																																																																																								
15	of	reserved	0	0	1	0	0	14																																																																																																																																																																																																								
16	or	reserved	0	0	1	0	0	15																																																																																																																																																																																																								
17	procedure	reserved	0	0	1	0	0	16																																																																																																																																																																																																								
18	program	reserved	0	0	1	0	0	17																																																																																																																																																																																																								
19	record	reserved	0	0	1	0	0	18																																																																																																																																																																																																								
20	repeat	reserved	0	0	1	0	0	19																																																																																																																																																																																																								

	21	integer	reserved	0	0	1	0	0	20
	22	real	reserved	0	0	1	0	0	21
	23	boolean	reserved	0	0	1	0	0	22
	24	char	reserved	0	0	1	0	0	23
	25	string	reserved	0	0	1	0	0	24
	26	then	reserved	0	0	1	0	0	25
	27	to	reserved	0	0	1	0	0	26
	28	type	reserved	0	0	1	0	0	27
	29	until	reserved	0	0	1	0	0	28
	30	var	reserved	0	0	1	0	0	29
	31	while	reserved	0	0	1	0	0	30
	32	Integer	type	32	0	1	0	0	31
	33	Real	type	33	0	1	0	0	32
	34	Boolean	type	34	0	1	0	0	33
	35	Char	type	35	0	1	0	0	34
	36	String	type	36	0	1	0	0	35
	37	True	constant	34	0	1	0	1	36
	38	False	constant	34	0	1	0	0	37
	39	Writeln	procedure	0	0	1	0	0	38
	40	Readln	procedure	0	0	1	0	0	39
	41	BooleanExpression	program	0	0	1	0	0	40
	42	a	variable	32	0	1	0	0	41
	43	b	variable	32	0	1	0	1	42
	44	ok	variable	34	0	1	0	2	43
	<pre> === btab (Block Table) === idx last lpar psze vsze ----- 0 44 0 0 3 === atab (Array Table) === (empty) === Errors === No errors found. === Warnings === No warnings.</pre>								
input-7.txt	<pre> === Decorated AST === ProgramNode(name: 'InvalidIfCondition') -> tab_index:41 ├── DeclarationListNode │ ├── DeclarationListNode │ │ └── VarDecl('x') -> tab_index:42, type:integer, lev:0 │ └── BlockNode -> block_index:0 │ ├── DeclarationListNode │ │ ├── Assign -> type:void │ │ │ ├── 'x' -> tab_index:42, type:Integer │ │ │ └── 5 -> type:Integer │ │ └── IfNode -> type:void │ │ └── 'x' -> tab_index:42, type:Integer </pre> <pre> === tab (Identifier Table) === idx id obj type ref nrm lev adr link ----- 0 and reserved 0 0 1 0 0 0 </pre>								

1	array	reserved	0	0	1	0	0	0
2	begin	reserved	0	0	1	0	0	1
3	case	reserved	0	0	1	0	0	2
4	const	reserved	0	0	1	0	0	3
5	div	reserved	0	0	1	0	0	4
6	downto	reserved	0	0	1	0	0	5
7	do	reserved	0	0	1	0	0	6
8	else	reserved	0	0	1	0	0	7
9	end	reserved	0	0	1	0	0	8
10	for	reserved	0	0	1	0	0	9
11	function	reserved	0	0	1	0	0	10
12	if	reserved	0	0	1	0	0	11
13	mod	reserved	0	0	1	0	0	12
14	not	reserved	0	0	1	0	0	13
15	of	reserved	0	0	1	0	0	14
16	or	reserved	0	0	1	0	0	15
17	procedure	reserved	0	0	1	0	0	16
18	program	reserved	0	0	1	0	0	17
19	record	reserved	0	0	1	0	0	18
20	repeat	reserved	0	0	1	0	0	19
21	integer	reserved	0	0	1	0	0	20
22	real	reserved	0	0	1	0	0	21
23	boolean	reserved	0	0	1	0	0	22
24	char	reserved	0	0	1	0	0	23
25	string	reserved	0	0	1	0	0	24
26	then	reserved	0	0	1	0	0	25
27	to	reserved	0	0	1	0	0	26
28	type	reserved	0	0	1	0	0	27
29	until	reserved	0	0	1	0	0	28
30	var	reserved	0	0	1	0	0	29
31	while	reserved	0	0	1	0	0	30
32	Integer	type	32	0	1	0	0	31
33	Real	type	33	0	1	0	0	32
34	Boolean	type	34	0	1	0	0	33
35	Char	type	35	0	1	0	0	34
36	String	type	36	0	1	0	0	35
37	True	constant	34	0	1	0	1	36
38	False	constant	34	0	1	0	0	37
39	Writeln	procedure	0	0	1	0	0	38
40	Readln	procedure	0	0	1	0	0	39
41	InvalidIfCondition	program	0	0	1	0	0	40
42	x	variable	32	0	1	0	0	41

=== btab (Block Table) ===

idx	last	lpar	psize	vsze
0	42	0	0	1

=== atab (Array Table) ===

(empty)

=== Errors ===

Error: If condition must be of type 'boolean', got 'Integer'

=== Warnings ===

No warnings.

input-8.txt

=== Decorated AST ===

```

ProgramNode(name: 'Scope') -> tab_index:41
├── DeclarationListNode
│   ├── DeclarationListNode
│   │   └── VarDecl('globalVar') -> tab_index:42, type:Integer, lev:0
│   └── procedure Inner -> tab_index:43, lev:0
│       ├── BlockNode -> block_index:1, lev:1
│       │   ├── DeclarationListNode
│       │   │   ├── DeclarationListNode
│       │   │   │   └── VarDecl('localVar') -> tab_index:44, type:Integer,
│       │   │   └── BlockNode -> block_index:2, lev:2
│       │       ├── DeclarationListNode
│       │       │   ├── Assign -> type:void, lev:2
│       │       │   │   ├── 'localVar' -> tab_index:44, type:Integer, lev:1
│       │       │   │   └── 10 -> type:Integer, lev:2
│       │       │   └── Assign -> type:void, lev:2
│       │       │   │   ├── 'globalVar' -> tab_index:42, type:Integer
│       │       │   │   └── BinOp '+' -> type:Integer, lev:2
│       │       │       ├── 'localVar' -> tab_index:44, type:Integer,
│       │       │       └── 1 -> type:Integer, lev:2
│       └── BlockNode -> block_index:0
│           ├── DeclarationListNode
│           │   ├── Assign -> type:void
│           │   │   ├── 'globalVar' -> tab_index:42, type:Integer
│           │   │   └── 0 -> type:Integer
│           │   ├── ProcCall(Inner) -> tab_index:43
│           │   └── ProcCall(writeln) -> tab_index:39
│           │       └── 'globalVar' -> tab_index:42, type:Integer
└── lev:1

```

=== tab (Identifier Table) ===

idx	id	obj	type	ref	nrm	lev	adr	link
0	and	reserved	0	0	1	0	0	0
1	array	reserved	0	0	1	0	0	0
2	begin	reserved	0	0	1	0	0	1
3	case	reserved	0	0	1	0	0	2
4	const	reserved	0	0	1	0	0	3
5	div	reserved	0	0	1	0	0	4
6	downto	reserved	0	0	1	0	0	5
7	do	reserved	0	0	1	0	0	6
8	else	reserved	0	0	1	0	0	7
9	end	reserved	0	0	1	0	0	8
10	for	reserved	0	0	1	0	0	9
11	function	reserved	0	0	1	0	0	10
12	if	reserved	0	0	1	0	0	11
13	mod	reserved	0	0	1	0	0	12
14	not	reserved	0	0	1	0	0	13
15	of	reserved	0	0	1	0	0	14
16	or	reserved	0	0	1	0	0	15
17	procedure	reserved	0	0	1	0	0	16
18	program	reserved	0	0	1	0	0	17
19	record	reserved	0	0	1	0	0	18
20	repeat	reserved	0	0	1	0	0	19
21	integer	reserved	0	0	1	0	0	20

	<pre> 22 real reserved 0 0 1 0 0 21 23 boolean reserved 0 0 1 0 0 22 24 char reserved 0 0 1 0 0 23 25 string reserved 0 0 1 0 0 24 26 then reserved 0 0 1 0 0 25 27 to reserved 0 0 1 0 0 26 28 type reserved 0 0 1 0 0 27 29 until reserved 0 0 1 0 0 28 30 var reserved 0 0 1 0 0 29 31 while reserved 0 0 1 0 0 30 32 Integer type 32 0 1 0 0 31 33 Real type 33 0 1 0 0 32 34 Boolean type 34 0 1 0 0 33 35 Char type 35 0 1 0 0 34 36 String type 36 0 1 0 0 35 37 True constant 34 0 1 0 1 36 38 False constant 34 0 1 0 0 37 39 Writeln procedure 0 0 1 0 0 38 40 Readln procedure 0 0 1 0 0 39 41 Scope program 0 0 1 0 0 40 42 globalVar variable 32 0 1 0 0 41 43 Inner procedure 0 0 1 0 0 42 44 localVar variable 32 0 1 1 0 0 </pre> <pre> === btab (Block Table) === idx last lpar psze vsze ----- 0 43 0 0 1 1 44 0 0 1 2 0 0 0 0 === atab (Array Table) === (empty) === Errors === No errors found. === Warnings === No warnings. </pre>
input-9.txt	<pre> === Decorated AST === ProgramNode(name: 'TestArray') -> tab_index:41 ├─ DeclarationListNode │ └─ DeclarationListNode │ └─ VarDecl('arr') -> tab_index:42, type:array[Integer] of Integer, lev:0 ├─ VarDecl('i') -> tab_index:43, type:Integer, lev:0 ├─ VarDecl('r') -> tab_index:44, type:Real, lev:0 └─ BlockNode -> block_index:0 └─ DeclarationListNode ├─ Assign -> type:void │ ├── 'i' -> tab_index:43, type:Integer │ └─ 3 -> type:Integer └─ Assign -> type:void ├── ArrayAccess -> type:unknown └─ 'arr' -> tab_index:42, type:unknown </pre>

```

└─ 'i' -> tab_index:43, type:Integer
└─ 10 -> type:Integer
└─ Assign -> type:void
└─ ArrayAccess -> type:unknown
└─ 'arr' -> tab_index:42, type:unknown
└─ 'r' -> tab_index:44, type:Real
└─ 5 -> type:Integer

```

=== tab (Identifier Table) ===

idx	id	obj	type	ref	nrm	lev	adr	link

0	and	reserved	0	0	1	0	0	0
1	array	reserved	0	0	1	0	0	0
2	begin	reserved	0	0	1	0	0	1
3	case	reserved	0	0	1	0	0	2
4	const	reserved	0	0	1	0	0	3
5	div	reserved	0	0	1	0	0	4
6	downto	reserved	0	0	1	0	0	5
7	do	reserved	0	0	1	0	0	6
8	else	reserved	0	0	1	0	0	7
9	end	reserved	0	0	1	0	0	8
10	for	reserved	0	0	1	0	0	9
11	function	reserved	0	0	1	0	0	10
12	if	reserved	0	0	1	0	0	11
13	mod	reserved	0	0	1	0	0	12
14	not	reserved	0	0	1	0	0	13
15	of	reserved	0	0	1	0	0	14
16	or	reserved	0	0	1	0	0	15
17	procedure	reserved	0	0	1	0	0	16
18	program	reserved	0	0	1	0	0	17
19	record	reserved	0	0	1	0	0	18
20	repeat	reserved	0	0	1	0	0	19
21	integer	reserved	0	0	1	0	0	20
22	real	reserved	0	0	1	0	0	21
23	boolean	reserved	0	0	1	0	0	22
24	char	reserved	0	0	1	0	0	23
25	string	reserved	0	0	1	0	0	24
26	then	reserved	0	0	1	0	0	25
27	to	reserved	0	0	1	0	0	26
28	type	reserved	0	0	1	0	0	27
29	until	reserved	0	0	1	0	0	28
30	var	reserved	0	0	1	0	0	29
31	while	reserved	0	0	1	0	0	30
32	Integer	type	32	0	1	0	0	31
33	Real	type	33	0	1	0	0	32
34	Boolean	type	34	0	1	0	0	33
35	Char	type	35	0	1	0	0	34
36	String	type	36	0	1	0	0	35
37	True	constant	34	0	1	0	1	36
38	False	constant	34	0	1	0	0	37
39	Writeln	procedure	0	0	1	0	0	38
40	Readln	procedure	0	0	1	0	0	39
41	TestArray	program	0	0	1	0	0	40
42	arr	variable	0	0	1	0	0	41
43	i	variable	32	0	1	0	1	42
44	r	variable	33	0	1	0	2	43

	<pre> === btab (Block Table) === idx last lpar psze vsze ----- 0 44 0 0 3 === atab (Array Table) === arrays xtyp etyp eref low high elsz size ----- 0 32 32 0 0 0 1 1 === Errors === No errors found. === Warnings === No warnings. </pre>
input-10.txt	<pre> === Decorated AST === ProgramNode(name: 'ComplexSemantic') -> tab_index:41 ├─ DeclarationListNode │ └─ DeclarationListNode │ ├── ConstDecl('limit') -> tab_index:42, type:Integer, lev:0 │ │ └─ 3 -> type:Integer │ └─ DeclarationListNode │ └─ TypeDecl('ScoreArray') -> tab_index:43, type:array[Integer] of integer, lev:0 │ └─ ArrayType -> type:array │ ├── Range -> type:subrange(Integer) │ │ ├── 1 -> type:Integer │ │ └─ 3 -> type:Integer │ └─ 'integer' -> tab_index:0, type:? └─ DeclarationListNode ├── VarDecl('scores') -> tab_index:44, type:ScoreArray, lev:0 ├── VarDecl('i') -> tab_index:45, type:integer, lev:0 ├── VarDecl('total') -> tab_index:46, type:integer, lev:0 ├── VarDecl('avg') -> tab_index:47, type:real, lev:0 └─ VarDecl('ok') -> tab_index:48, type:boolean, lev:0 └─ BlockNode -> block_index:0 └─ DeclarationListNode ├── Assign -> type:void │ ├── 'total' -> tab_index:46, type:Integer │ └─ 0 -> type:Integer ├── Assign -> type:void │ ├── 'i' -> tab_index:45, type:Integer │ └─ 1 -> type:Integer ├── WhileNode -> type:void │ ├── BinOp '<=' -> type:Boolean │ │ ├── 'i' -> tab_index:45, type:Integer │ │ └─ 3 -> type:Integer │ └─ BlockNode -> block_index:0 │ └─ DeclarationListNode │ └─ Assign -> type:void │ ├── ArrayAccess -> type:unknown │ └─ 'scores' -> tab_index:44, type:unknown </pre>

									└─ 'i' -> tab_index:45, type:Integer └─ BinOp '*' -> type:Integer └─ 'i' -> tab_index:45, type:Integer └─ 10 -> type:Integer └─ Assign -> type:void └─ 'total' -> tab_index:46, type:Integer └─ BinOp '+' -> type:unknown └─ 'total' -> tab_index:46, type:Integer └─ ArrayAccess -> type:unknown └─ 'scores' -> tab_index:44, type:unknown └─ 'i' -> tab_index:45, type:Integer └─ Assign -> type:void └─ 'i' -> tab_index:45, type:Integer └─ BinOp '+' -> type:Integer └─ 'i' -> tab_index:45, type:Integer └─ 1 -> type:Integer
									└─ Assign -> type:void └─ 'avg' -> tab_index:47, type:Real └─ BinOp '/' -> type:Real └─ 'total' -> tab_index:46, type:Integer └─ 3 -> type:Integer └─ Assign -> type:void └─ 'ok' -> tab_index:48, type:Boolean └─ BinOp '>=' -> type:Boolean └─ 'avg' -> tab_index:47, type:Real └─ 20 -> type:Real └─ IfNode -> type:void └─ 'ok' -> tab_index:48, type:Boolean
									<pre> === tab (Identifier Table) === idx id obj type ref nrm lev adr link ----- 0 and reserved 0 0 1 0 0 0 1 array reserved 0 0 1 0 0 0 2 begin reserved 0 0 1 0 0 1 3 case reserved 0 0 1 0 0 2 4 const reserved 0 0 1 0 0 3 5 div reserved 0 0 1 0 0 4 6 downto reserved 0 0 1 0 0 5 7 do reserved 0 0 1 0 0 6 8 else reserved 0 0 1 0 0 7 9 end reserved 0 0 1 0 0 8 10 for reserved 0 0 1 0 0 9 11 function reserved 0 0 1 0 0 10 12 if reserved 0 0 1 0 0 11 13 mod reserved 0 0 1 0 0 12 14 not reserved 0 0 1 0 0 13 15 of reserved 0 0 1 0 0 14 16 or reserved 0 0 1 0 0 15 17 procedure reserved 0 0 1 0 0 16 18 program reserved 0 0 1 0 0 17 19 record reserved 0 0 1 0 0 18 20 repeat reserved 0 0 1 0 0 19 21 integer reserved 0 0 1 0 0 20 22 real reserved 0 0 1 0 0 21 23 boolean reserved 0 0 1 0 0 22 </pre>

24	char	reserved	0	0	1	0	0	23
25	string	reserved	0	0	1	0	0	24
26	then	reserved	0	0	1	0	0	25
27	to	reserved	0	0	1	0	0	26
28	type	reserved	0	0	1	0	0	27
29	until	reserved	0	0	1	0	0	28
30	var	reserved	0	0	1	0	0	29
31	while	reserved	0	0	1	0	0	30
32	Integer	type	32	0	1	0	0	31
33	Real	type	33	0	1	0	0	32
34	Boolean	type	34	0	1	0	0	33
35	Char	type	35	0	1	0	0	34
36	String	type	36	0	1	0	0	35
37	True	constant	34	0	1	0	1	36
38	False	constant	34	0	1	0	0	37
39	Writeln	procedure	0	0	1	0	0	38
40	Readln	procedure	0	0	1	0	0	39
41	ComplexSemantic	program	0	0	1	0	0	40
42	limit	constant	32	0	1	0	3	41
43	ScoreArray	type	0	0	1	0	0	42
44	scores	variable	0	0	1	0	0	43
45	i	variable	32	0	1	0	1	44
46	total	variable	32	0	1	0	2	45
47	avg	variable	33	0	1	0	3	46
48	ok	variable	34	0	1	0	4	47

=== btab (Block Table) ===

idx	last	lpar	psze	vsze

0	48	0	0	5

=== atab (Array Table) ===

arrays	xtyp	etyp	eref	low	high	elsz	size

0	32	32	0	0	0	1	1

=== Errors ===

Error: Operator '+' not defined for types 'Integer' and 'unknown'

=== Warnings ===

No warnings.

BAB 4

Kesimpulan dan Saran

4.1 Kesimpulan

Pada Milestone 3 Tugas Besar IF2224 Teori Bahasa Formal dan Otomata, kami telah berhasil mengimplementasikan tahap semantic analysis pada compiler Arion. Tahap ini telah terintegrasi dengan lexer dan parser, sehingga parse tree hasil parser dapat diproses menjadi Decorated AST, symbol table, serta hasil pengecekan semantik.

Decorated AST diimplementasikan dengan atribut semantik seperti type, level, dan tabRef. Symbol table diimplementasikan menggunakan tab, btab, dan atab untuk menyimpan identifier, scope, serta informasi array. Selain itu, predefined identifier seperti Integer, Real, Boolean, Char, String, True, False, Writeln, dan Readln juga telah diinisialisasi.

Semantic analyzer telah mendukung pengecekan deklarasi dan duplikasi identifier, lookup berdasarkan scope, inferensi tipe, validasi assignment dan operator, serta pengecekan kondisi Boolean pada struktur kontrol. Hasil semantic analysis dapat ditampilkan melalui mode --semantic dalam bentuk Decorated AST, symbol table, error, dan warning.

4.2 Saran

Kami ingin berbagi beberapa saran kepada pembaca yang mungkin akan menghadapi tugas serupa di masa depan:

BillyO	rajin - rajin cek qna
Nathan	Cukup sudah cukup
Azri	Kapan ya sejago Sandwicheese
Rasyad	Kak william tolong ini nya di anuin

Lampiran

Repository: <https://github.com/AzriVz/CTF-Tubes-IF2224-2026>

Diagram Transisi DFA:

<https://drive.google.com/file/d/1Q8MlnukMm2jU2dEt5GNVJjzSQ2MCpIng/view?usp=sharing>

Tabel Pembagian Tugas:

Nama	NIM	Workload	Persentase
Billy Ontoseno Irawan	13524121	Laporan (Bab 1)	22%
Nathaniel Christian	13524122	Laporan (Bab 2 dan 4)	22%
Azri Arzaq Pohan	13523139	Laporan (Bab 3) Test cases	21%
Rasyad Satyatma	13524142	./src/Semantic (source code)	35%

Referensi

[CS 241 Lecture 9 - Maximal Munch and Context Free Grammars With thanks to Brad Lushman, Troy Vasiga and Kevin Lanctot](#)

[CS 241 Lecture 11 - Top Down Parsing, First and Follow With thanks to Brad Lushman, Troy Vasiga and Kevin Lanctot](#)